Lecture 2. Data Visualisation

ggplot2: aesthetics, facets, geoms, stats, positions, coordinates, graphics for communication

Contents

2.1 Data visualisation with ggplot2 **2**
- 2.1.1 The mpg data frame . . . 2
- 2.1.2 Creating a ggplot . . . 2

2.2 Aesthetic mappings **3**

2.3 Facets **4**

2.4 Geometric objects **4**

2.5 Statistical transformations **5**

2.6 Position adjustments **6**

2.7 Coordinate systems **8**

2.8 The layered grammar of graphics **8**

2.9 Graphics for communication **9**
- 2.9.1 Labels . . . 9
- 2.9.2 Annotations . . . 9
- 2.9.3 Scales, legends and zooming . . . 10
- 2.9.4 Themes and saving . . . 10

2.1 Data visualisation with ggplot2

Motivation. Visualisation is the best place to start programming with R because the payoff is immediate: a few lines of code turn a table nobody can read into a picture that answers a question. ggplot2 builds every plot from the same small set of parts, so learning those parts once covers almost every chart you will need.

Throughout we load the tidyverse, which attaches `ggplot2`:

```
library(tidyverse)
```

2.1.1 The mpg data frame

Do cars with big engines use more fuel than cars with small ones, and what does the relationship look like (positive or negative, linear or not)? The data frame `mpg` in ggplot2 (`ggplot2::mpg`) records 234 cars; `displ` is engine size in litres and `hwy` highway fuel efficiency in miles per gallon.

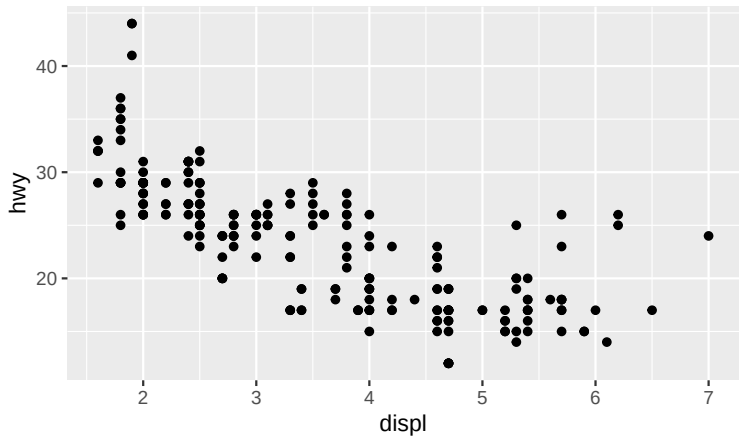
```
mpg
```

```
## # A tibble: 234 x 11
##    manufacturer model      displ  year   cyl trans      drv     cty   hwy fl    class
##    <chr>        <chr>      <dbl> <int> <int> <chr>      <chr> <int> <int> <chr> <chr>
##  1 audi         a4           1.8  1999     4 auto(l5)   f        18    29 p     compact
##  2 audi         a4           1.8  1999     4 manual(m5) f        21    29 p     compact
##  3 audi         a4           2    2008     4 manual(m6) f        20    31 p     compact
##  4 audi         a4           2    2008     4 auto(av)   f        21    30 p     compact
##  5 audi         a4           2.8  1999     6 auto(l5)   f        16    26 p     compact
##  6 audi         a4           2.8  1999     6 manual(m5) f        18    26 p     compact
##  7 audi         a4           3.1  2008     6 auto(av)   f        18    27 p     compact
##  8 audi         a4 quattro   1.8  1999     4 manual(m5) 4        18    26 p     compact
##  9 audi         a4 quattro   1.8  1999     4 auto(l5)   4        16    25 p     compact
## 10 audi         a4 quattro   2    2008     4 manual(m6) 4        20    28 p     compact
## # i 224 more rows
```

A **data frame** is a rectangular collection of variables (columns) and observations (rows); `mpg` is a tibble, the tidyverse version of a data frame (Lecture 5).

2.1.2 Creating a ggplot

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy))
```



The plot shows a negative relationship: bigger engines, lower efficiency. The code has three parts.

- `ggplot(data = mpg)` creates an empty coordinate system whose data set is `mpg`; on its own it draws a blank canvas.
- `geom_point()` adds a **layer** of points. Layers are added with `+`.
- The `mapping` argument says how variables are mapped to visual properties. It is always given by `aes()`; here `x = displ` and `y = hwy`. ggplot2 looks up `displ` and `hwy` inside the data set.

Every ggplot follows the **graphing template**

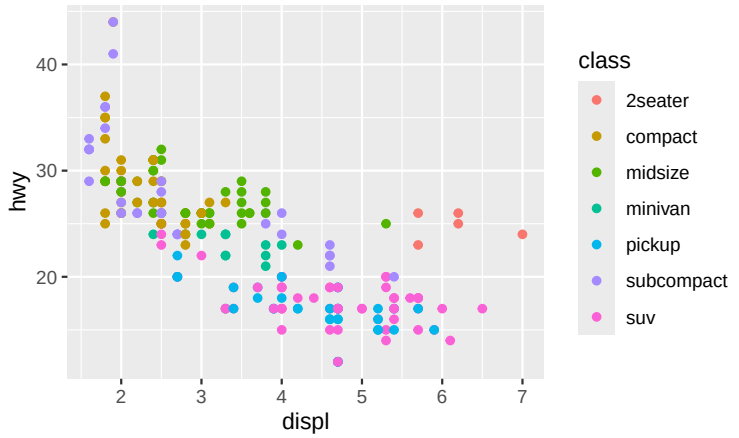
```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(mapping = aes(<MAPPINGS>))
```

2.2 Aesthetic mappings

Motivation. The scatterplot leaves something unexplained: a group of cars with large engines but unusually high mileage. To explain them we need a third variable, and aesthetics are how a third (or fourth) variable enters a two-dimensional plot.

An **aesthetic** is a visual property of the objects in a plot: position, size, shape, colour, transparency. Mapping a variable to an aesthetic makes that property vary with the variable. Mapping `class` to colour reveals that the unusual cars are two-seaters (sports cars with light bodies):

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy, color = class))
```



ggplot2 assigns a level of the aesthetic to each value (**scaling**) and adds a legend explaining the mapping. Other aesthetics are used in the same way:

```
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, size = class))
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, alpha = class))
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy, shape = class))
```

- `size` and `alpha` (transparency) imply an order, so mapping an unordered variable such as `class` to them is not advised; ggplot2 warns about it.
- `shape` uses at most six shapes at a time; further groups are silently dropped (here SUVs disappear). R has 25 built-in shapes, numbered 0–24; shapes 21–24 have both a border (`colour`) and an interior (`fill`).

Setting versus mapping. To give *all* points the same property, set the aesthetic as an argument of the geom, *outside* `aes()`:

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy), color = "blue")
```

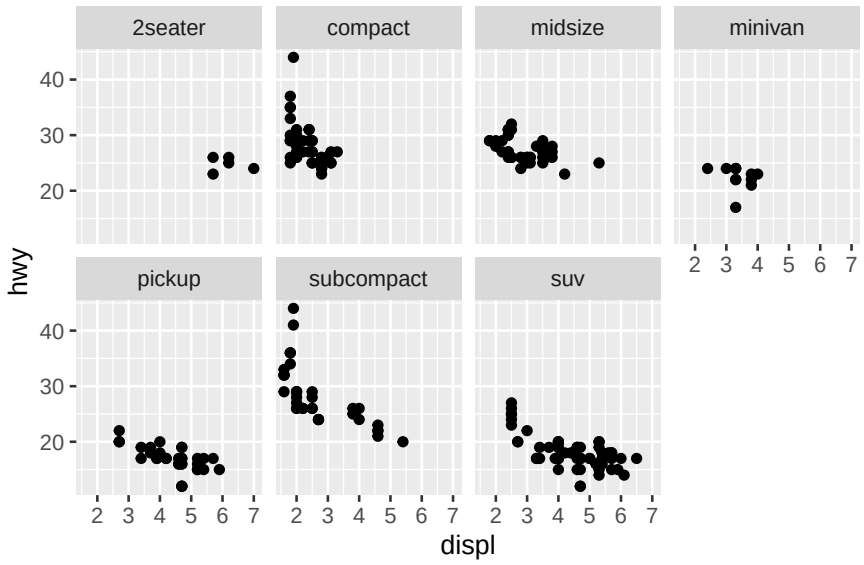
Writing `aes(color = "blue")` instead would map the constant string `"blue"` to colour, producing one group called "blue" drawn in the default first colour (red) with a legend.

2.3 Facets

Motivation. Aesthetics crowd a plot when the extra variable has many levels. The alternative is to draw one small plot per level and compare them side by side.

`facet_wrap()` splits the plot by one discrete variable, given as a formula:

```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  facet_wrap(~ class, nrow = 2)
```



`facet_grid(drv ~ cyl)` uses two variables, rows by the first and columns by the second; `facet_grid(. ~ cyl)` facets by columns only. Facets make each group easy to see on its own, at the cost of harder comparisons between groups than colour gives.

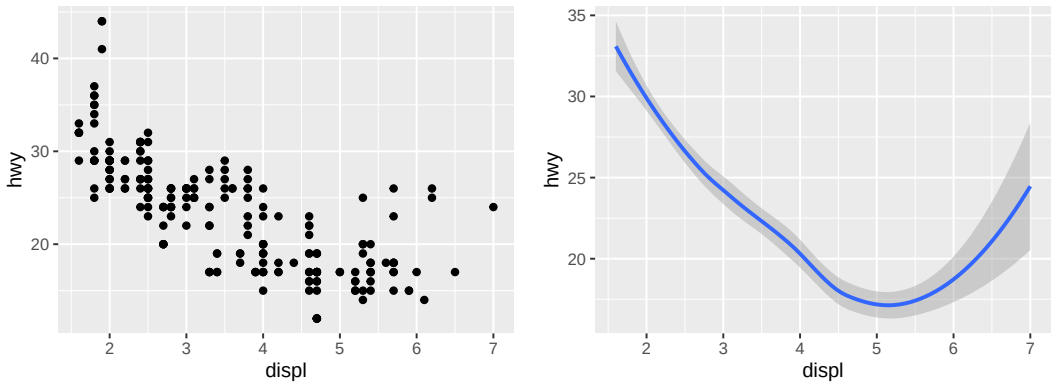
```
ggplot(data = mpg) +
  geom_point(mapping = aes(x = displ, y = hwy)) +
  facet_grid(drv ~ class)
```

2.4 Geometric objects

Motivation. The same data can be drawn as points, as a smooth curve, as bars or as boxes. In ggplot2 this choice is a single function, the geom, while the data and mapping stay the same.

A **geom** is the geometrical object a plot uses to represent data. The two plots below use the same data and mapping but different geoms:

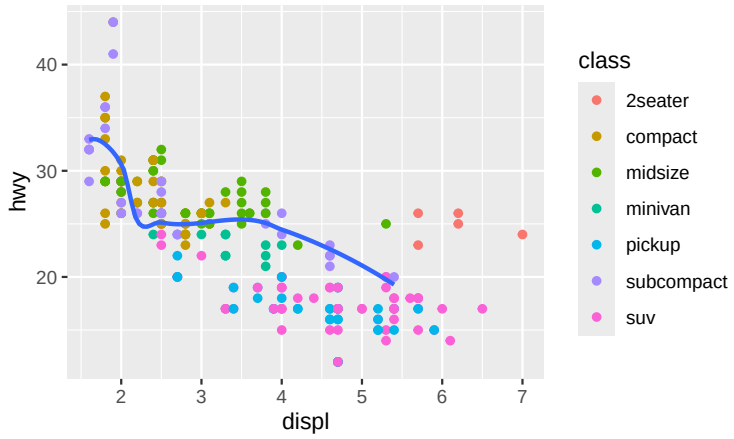
```
ggplot(data = mpg) + geom_point(mapping = aes(x = displ, y = hwy))
ggplot(data = mpg) + geom_smooth(mapping = aes(x = displ, y = hwy))
```



Every geom takes a `mapping`, but not every aesthetic applies to every geom: lines have a `linetype`, points do not. `geom_smooth(aes(linetype = drv))` draws one curve per drive train with different line types, and `aes(color = drv)` with different colours. Grouping happens automatically whenever a discrete variable is mapped; `aes(group = drv)` groups without adding a legend.

Several geoms can be layered. Mappings given in `ggplot()` are **global** and apply to every layer; mappings given in a geom are **local** to that layer and extend or override the global ones. A layer can even use its own data:

```
ggplot(data = mpg, mapping = aes(x = displ, y = hwy)) +
  geom_point(mapping = aes(color = class)) +
  geom_smooth(data = filter(mpg, class == "subcompact"), se = FALSE)
```



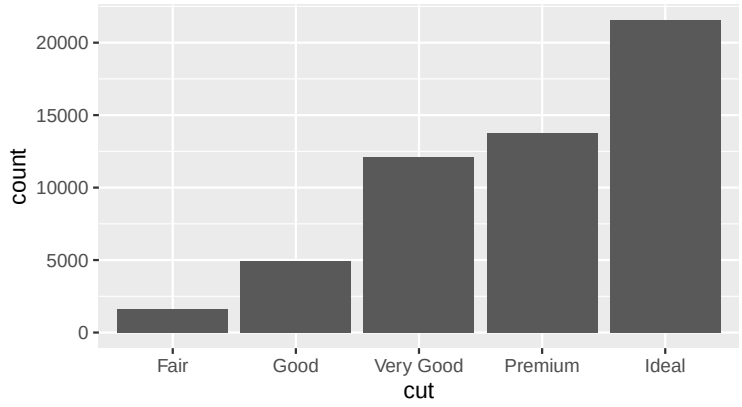
Here the smooth line is fitted to subcompact cars only (`filter()` is introduced in Lecture 3) and `se = FALSE` removes the confidence band. Without the `data` argument the line would be fitted to all cars.

2.5 Statistical transformations

Motivation. A bar chart of diamond cuts shows counts, yet the data contain no count column. Some plots draw values computed from the data rather than the data themselves; seeing where those values come from lets us change them.

The `diamonds` data set contains about 54 000 diamonds with their price, carat, cut, colour and clarity.

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut))
```

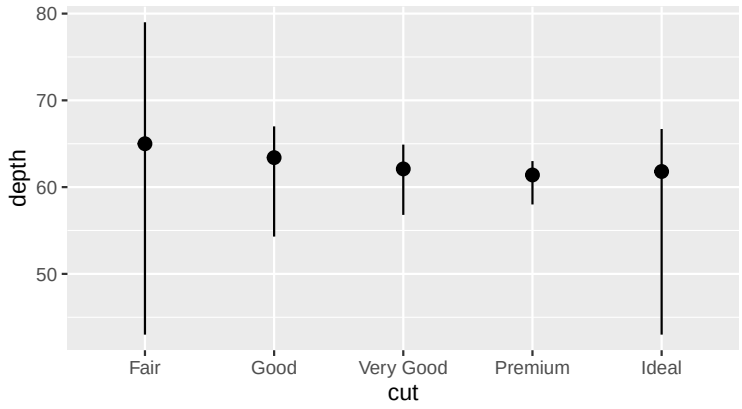


The y-axis, `count`, is not a variable of `diamonds`. Bar charts, histograms and frequency polygons bin the data and count; smoothers fit a model; boxplots compute summaries. The algorithm that computes the new values is a **stat**. The help page `?geom_bar` shows that its default stat is `"count"`, i.e. `stat_count()`; conversely `stat_count()` has default geom `geom_bar()`, so `ggplot(diamonds) + stat_count(aes(x = cut))` draws the same plot.

Reasons to use a stat explicitly:

- to plot a different computed variable, e.g. proportions instead of counts: `aes(x = cut, y = after_stat(prop), group = 1)`. The `group = 1` is needed because otherwise each bar is its own group and every proportion equals 1;
- to plot a summary of a variable for each x, e.g. the median depth with its range:

```
ggplot(data = diamonds) +
  stat_summary(
    mapping = aes(x = cut, y = depth),
    fun.min = min, fun.max = max, fun = median
  )
```

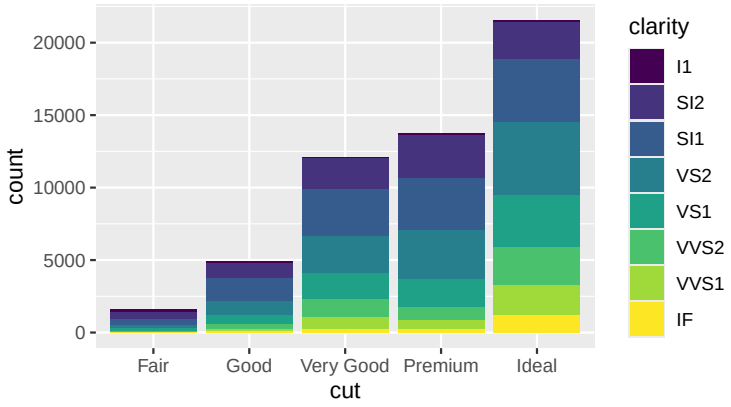


2.6 Position adjustments

Motivation. When bars or points share a position they overlap. How overlapping objects are arranged changes what the plot shows, so it is a deliberate choice rather than a default to accept.

Mapping a second variable to `fill` stacks the bars automatically:

```
ggplot(data = diamonds) +
  geom_bar(mapping = aes(x = cut, fill = clarity))
```



The heights of the coloured segments are the counts of each clarity within the cut, e.g. for ideal diamonds:

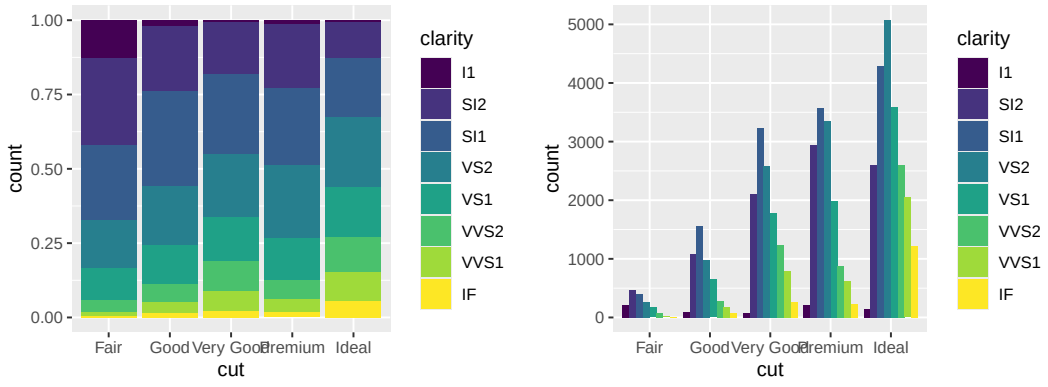
```
diamonds %>% filter(cut == "Ideal") %>% count(clarity)
```

```
## # A tibble: 8 x 2
##   clarity     n
##   <ord>   <int>
## 1 I1        146
## 2 SI2      2598
## 3 SI1      4282
## 4 VS2      5071
## 5 VS1      3589
## 6 VVS2     2606
## 7 VVS1     2047
## 8 IF       1212
```

The `position` argument controls the arrangement:

- `"stack"` (default for bars): segments on top of each other;
- `"identity"`: each bar starts at zero, so bars overlap (useful with `alpha` or for lines, not for bars);
- `"fill"`: stacked and rescaled to height 1, which compares *proportions* across groups;
- `"dodge"`: bars side by side, which compares individual counts.

```
ggplot(diamonds) + geom_bar(aes(x = cut, fill = clarity), position = "fill")
ggplot(diamonds) + geom_bar(aes(x = cut, fill = clarity), position = "dodge")
```



For scatterplots, `mpg` has 234 cars but the plot shows fewer points: the values are rounded, so many points coincide (**overplotting**). `position = "jitter"` adds a little random noise to each point.

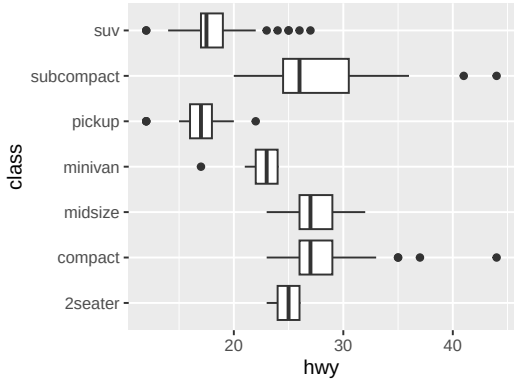
This makes the plot less accurate at small scales but more revealing at large scales; `geom_jitter()` is a shorthand.

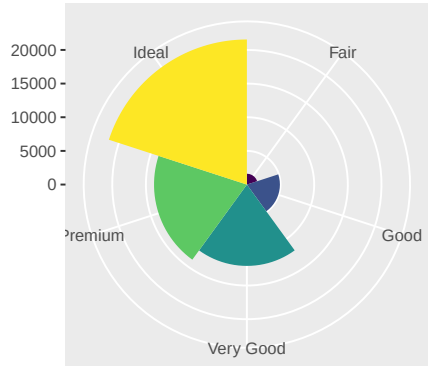
2.7 Coordinate systems

The default is Cartesian (`coord_cartesian()`, which also zooms with `xlim`, `ylim`). Others:

- `coord_flip()` swaps the axes, e.g. for horizontal boxplots with long labels;
- `coord_fixed(ratio = )` fixes the aspect ratio;
- `coord_quickmap()` sets the aspect ratio correctly for maps (`map_data("nz")` from the `maps` package gives map polygons for `geom_polygon()`);
- `coord_polar()` uses polar coordinates, which shows that a bar chart and a pie (Coxcomb) chart are the same graph in different coordinates.

```
ggplot(data = mpg, mapping = aes(x = class, y = hwy)) +
  geom_boxplot() + coord_flip()
bar <- ggplot(data = diamonds) +
  geom_bar(aes(x = cut, fill = cut), show.legend = FALSE, width = 1) +
  theme(aspect.ratio = 1) + labs(x = NULL, y = NULL)
bar + coord_polar()
```





2.8 The layered grammar of graphics

Motivation. All the pieces seen so far fit into one template. Knowing the template means any plot can be described by filling in its slots.

```
ggplot(data = <DATA>) +
  <GEOM_FUNCTION>(
     mapping = aes(<MAPPINGS>),
     stat = <STAT>,
     position = <POSITION>
  ) +
  <COORDINATE_FUNCTION> +
  <FACET_FUNCTION>
```

The seven parameters (data, mapping, geom, stat, position, coordinates, facets) describe every plot: start with a data set, transform it with a stat into the information to display, choose a geom to represent each value, map variables to aesthetics, place the geoms in a coordinate system and, optionally, split into facets.

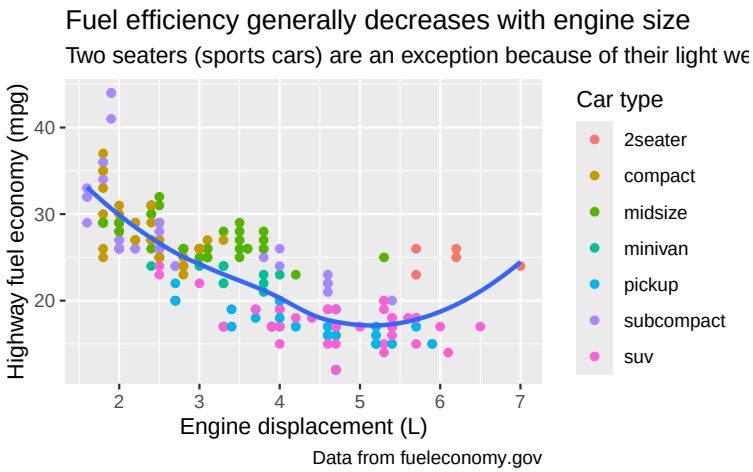
2.9 Graphics for communication

Motivation. Plots made during exploration are for you; plots in a report are for readers who do not know the data. Those need titles, readable labels and careful scales.

2.9.1 Labels

`labs()` sets the `title` (state the main finding, not just what is plotted), `subtitle`, `caption` (often the data source), axis labels and legend titles:

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(color = class)) +
  geom_smooth(se = FALSE) +
  labs(
    title = "Fuel efficiency generally decreases with engine size",
    subtitle = "Two seaters (sports cars) are an exception because of their light weight
    ",
    caption = "Data from fueleconomy.gov",
    x = "Engine displacement (L)", y = "Highway fuel economy (mpg)", colour = "Car type"
  )
```



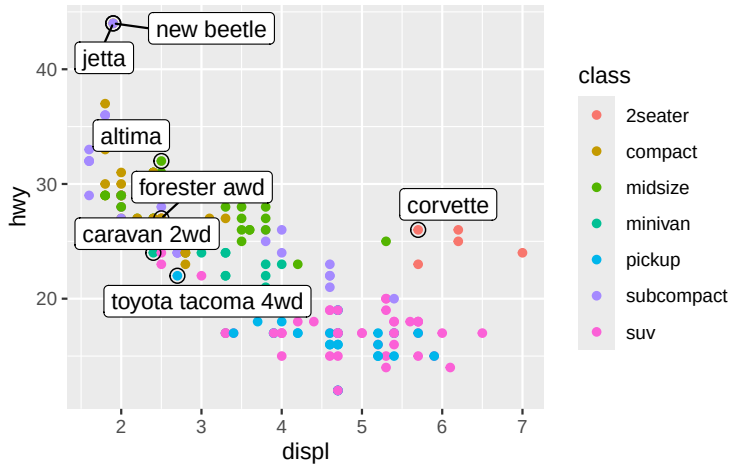
Axis labels can contain mathematical expressions written with `quote()`, e.g. `labs(x = quote(sum(x[i]^2, i == 1, n)))`; see `?plotmath`.

2.9.2 Annotations

To label individual observations, use `geom_text()` with its own data. For example, the most efficient car in each class:

```
best_in_class <- mpg %>%
  group_by(class) %>%
  filter(row_number(desc(hwy)) == 1)
```

```
ggplot(mpg, aes(displ, hwy)) +
  geom_point(aes(colour = class)) +
  geom_point(size = 3, shape = 1, data = best_in_class) +
  ggrepel::geom_label_repel(aes(label = model), data = best_in_class)
```



Plain `geom_text()` lets labels overlap each other and the points; `ggrepel::geom_label_repel()` moves them apart automatically.

2.9.3 Scales, legends and zooming

ggplot2 adds default scales behind the scenes: `scale_x_continuous()`, `scale_y_continuous()`, `scale_colour_discrete()`. Overriding them changes

- the **breaks** (tick positions): `scale_y_continuous(breaks = seq(15, 40, by = 5))`;
- the **labels**: `labels = NULL` removes them;
- the **transformation**: `scale_y_log10()`, `scale_x_reverse()`.

The legend position is set by `theme(legend.position = "bottom")` (or `"left"`, `"top"`, `"right"`, `"none"`).

To zoom there are two different operations:

- `coord_cartesian(xlim = c(5, 7), ylim = c(10, 30))` zooms the view *without removing data*, so the smoother is still fitted to all cars;
- `xlim(5, 7)`, `scale_x_continuous(limits = c(5, 7))`, or filtering the data first remove the points outside the range, so the smoother changes.

2.9.4 Themes and saving

Themes change the non-data elements (background, grid lines, fonts): `theme_bw()`, `theme_classic()`, `theme_minimal()` and others; ggplot2 has eight built-in themes. `ggsave("my-plot.pdf")` saves the most recent plot (size taken from the current device unless `width` and `height` are given).

ggplot2 has more than 30 geoms, and extension packages add many more; the ggplot2 cheat sheet is the best overview.